TECNOLÓGICO DE ESTUDIOS SUPERIORES DE ECATEPEC

DIVISIÓN DE INGENIERÍA EN SISTEMAS COMPUTACIONALES

PROGRAMACIÓN DE BASE DE DATOS

TAREA 8: PROGRAMACIÓN DEL LADO DEL SERVIDOR: CREACIÓN DE FUNCIONES

PROFESORA:

ALEJANDRA GONZALEZ SÁNCHEZ

NOMBRE:

MONTERRUBIO LÓPEZ LUIS GAEL

GRUPO: 5701

FECHA: 30-10-26

1. Introducción y objetivo

Este informe explica qué son las funciones en una base de datos relacional: su propósito, sus parámetros, sus valores de retorno y sus tipos. Después describe el diseño, la implementación, las pruebas y el análisis de rendimiento de las funciones creadas en PostgreSQL para el proyecto IoT Analytics Dashboard de NexoDev Solutions.

La **función principal del entregable** es `fn_registrar_lectura`. Con una sola llamada encapsula toda la operación de registrar una lectura de un sensor: valida que el sensor exista, que su dispositivo esté activo y que el valor esté en un rango físico posible; guarda la lectura y, si supera el umbral, **genera la alerta** correspondiente. La acompañan funciones de apoyo reutilizables, entre ellas `fn_promedio_sensor`, que **calcula promedios en tiempo real**: son los dos ejemplos de operación específica que plantea la actividad.

Las funciones se integran a un entorno IoT simulado con Docker: un simulador publica lecturas en RabbitMQ, un consumidor en Python las registra llamando a `fn_registrar_lectura` con SQL embebido, y Grafana construye su tablero llamando a las funciones de consulta.

Repositorio del proyecto: https://github.com/luisgael-ml/iot-funciones

Competencia específica: manejar un lenguaje de programación visual y la conexión a la base de datos mediante código SQL embebido (clave de actividad BDM-1301T03CE01A03).

Objetivos específicos:

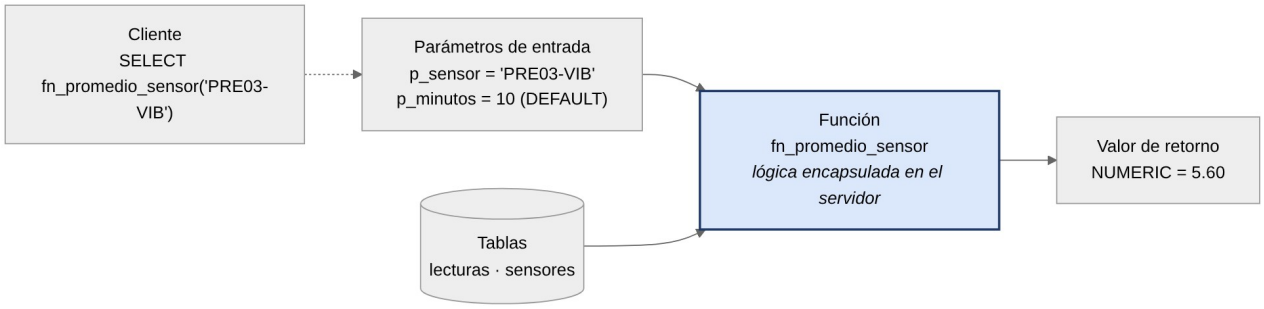
1. Diseñar la lógica de una función en SQL que encapsule una operación específica: el cálculo de promedios y la generación de alertas.
2. Implementar las funciones en PostgreSQL y probarlas con datos de prueba representativos.
3. Configurar un entorno simulado con Docker para verificar la integración de las funciones con un sistema IoT.
4. Documentar el proceso de desarrollo, el código, las pruebas realizadas y el análisis de rendimiento.
5. Presentar las funciones implementadas y su impacto en la optimización de operaciones del IoT Analytics Dashboard.

2. Marco teórico: funciones en bases de datos

2.1 ¿Qué es una función?

Una función es un bloque de código **guardado y ejecutado dentro del servidor de base de datos**. Recibe valores de entrada llamados parámetros, realiza una operación y **devuelve un resultado**. Se crea una sola vez con `CREATE FUNCTION` y después cualquier usuario con permiso la llama por su nombre, por ejemplo `SELECT fn_promedio_sensor('PRE03-VIB')`, sin conocer las tablas ni la lógica interna.

Programar del lado del servidor significa llevar la lógica a donde están los datos: en vez de traer miles de filas a la aplicación para procesarlas, la aplicación envía una llamada y recibe solo el resultado.



Una función recibe parámetros, opera sobre las tablas en el servidor y devuelve un valor.

2.2 Propósito y ventajas

PROPÓSITO	DESCRIPCIÓN	EN ESTE PROYECTO
Encapsular lógica reutilizable	La regla se escribe una vez y la usan todos los clientes	`fn_clasificar_lectura` la reutilizan 3 funciones
Rendimiento	Los datos se procesan en el servidor; por la red solo viaja el resultado	Grafana recibe un promedio, no miles de lecturas
Integridad	Las validaciones se cumplen sin importar qué cliente inserte	`fn_registrar_lectura` rechaza sensores inexistentes y valores imposibles
Seguridad	Se da permiso de ejecutar la función, no de tocar las tablas	El consumidor no puede hacer INSERT directo
Atomicidad	Una función se ejecuta dentro de una transacción: todo o nada	La lectura y su alerta se guardan juntas

Desventajas: la lógica queda ligada al gestor (PL/pgSQL no es portable a otros motores); depurar y versionar código dentro de la base de datos es más difícil que en una aplicación; y una función mal escrita, por ejemplo con ciclos fila por fila, puede ser mucho más lenta que una consulta equivalente.

2.3 Parámetros

TIPO DE PARÁMETRO	USO	EJEMPLO EN EL PROYECTO
IN (entrada)	Valor que la función recibe; es el modo por defecto	`p_sensor TEXT` en `fn_promedio_sensor`
Con valor por defecto (DEFAULT)	Se puede omitir al llamar a la función	`p_minutos INT DEFAULT 10`; `p_medido_en TIMESTAMPTZ DEFAULT NOW()`
OUT (salida)	Valores que la función devuelve; varios OUT forman un registro	`fn_estadisticas_sensor` devuelve 7 parámetros OUT
Tipados	Cada parámetro tiene un tipo; PostgreSQL elige la función por nombre y tipos (sobrecarga)	`NUMERIC`, `TEXT`, `TIMESTAMPTZ`, `INTERVAL`

2.4 Valores de retorno y tipos de funciones

TIPO	DEVUELVE	CÓMO SE LLAMA	FUNCIÓN DEL PROYECTO
Escalar	Un solo valor	`SELECT fn(...)`	`fn_clasificar_lectura` (TEXT), `fn_valor_valido` (BOOLEAN), `fn_promedio_sensor` (NUMERIC)
De registro	Una fila con varias columnas (parámetros	`SELECT * FROM fn(...)`	`fn_estadisticas_sensor`

TIPO	DEVUELVE	CÓMO SE LLAMA	FUNCIÓN DEL PROYECTO
	OUT)		
De tabla (RETURNS TABLE)	Un conjunto de filas; se usa como una tabla	`SELECT * FROM fn(...)`	`fn_registrar_lectura`, `fn_serie_promedios`, `fn_prediccion`
Disparador (RETURNS trigger)	Se ejecuta sola ante un INSERT, UPDATE o DELETE	Automática	Se usó en las Tareas 5 y 7; aquí se reemplaza por una llamada explícita

2.5 Lenguajes: SQL y PL/pgSQL

ASPECTO	LANGUAGE SQL	LANGUAGE PLPGSQL
Contenido	Una o varias consultas SQL	Variables, IF, ciclos, manejo de errores
Rendimiento	El planificador puede integrarla a la consulta que la llama (inlining)	Se ejecuta paso a paso; sus planes se guardan en caché
Cuándo usarlo	Cálculos y consultas	Validaciones con varios pasos y mensajes de error
En el proyecto	F1, F2, F4, F5, F6 y F7	F3 (`fn_registrar_lectura`) y F8 (versión de prueba con ciclo)

2.6 Volatilidad, seguridad y errores

CONCEPTO	SIGNIFICADO	USO EN EL PROYECTO
IMMUTABLE	Mismo resultado para los mismos parámetros, sin leer tablas; el planificador puede precalcularla	`fn_clasificar_lectura`, `fn_valor_valido`
STABLE	Lee tablas pero no las modifica; dentro de una consulta devuelve lo mismo	Funciones de consulta F4 a F7
VOLATILE	Puede modificar datos o cambiar en cada llamada (valor por defecto)	`fn_registrar_lectura` (inserta)
SECURITY DEFINER	Se ejecuta con los permisos de su dueño, no de quien la llama	El consumidor y Grafana no necesitan permisos sobre tablas
SET search_path	Fija dónde buscar las tablas; evita que un usuario suplante objetos	Todas las funciones SECURITY DEFINER
RAISE EXCEPTION	Detiene la función, deshace la transacción y devuelve un error con código	Códigos P0002, 55000 y 22003 en F3

2.7 Función y procedimiento almacenado

ASPECTO	FUNCIÓN (FUNCTION)	PROCEDIMIENTO (PROCEDURE)
Devuelve un valor	Sí, siempre	No (solo parámetros OUT/INOUT)
Se llama con	`SELECT`, dentro de cualquier consulta	`CALL`
Control de transacciones	No puede hacer COMMIT	Puede hacer COMMIT y ROLLBACK
Uso típico	Cálculos, consultas y operaciones que devuelven un resultado	Procesos por lotes largos

Se eligieron funciones porque cada operación debe **devolver un resultado** al cliente (la situación de la lectura, un promedio o una tabla para Grafana) y porque se pueden usar dentro de un `SELECT`.

2.8 Ejemplos en sistemas reales

SISTEMA	FUNCIÓN	PARA QUÉ
Banca	`calcular_interes(cuenta, fecha)`	La misma regla de intereses para cajeros, app y estados de cuenta
Comercio electrónico	`precio_final(producto, cliente)`	Aplica descuentos e impuestos en un solo lugar
ERP (SAP)	Funciones AMDP / SQLScript en SAP HANA	Cálculos de finanzas ejecutados dentro de la base de datos
Geolocalización	`ST_Distance` de PostGIS	Distancias calculadas en el servidor
IoT Analytics Dashboard	`fn_registrar_lectura`, `fn_promedio_sensor`	Validar y alertar en tiempo real; promedios para el tablero

3. Análisis de requisitos

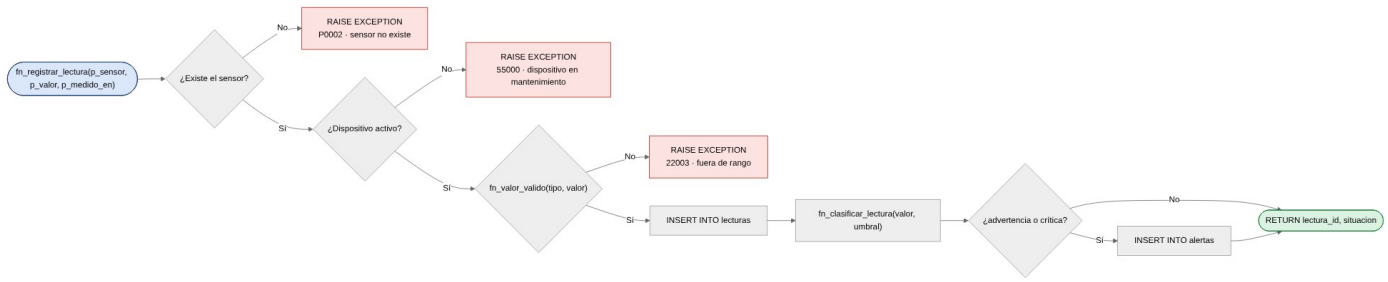
Antes de programar se definieron las operaciones que el sistema IoT necesita y la función que cubre cada una.

NO.	REQUISITO	TIPO	FUNCIÓN
R1	Registrar cada lectura validando sensor, estado del dispositivo y rango físico	Operación principal	`fn_registrar_lectura`
R2	Generar una alerta de advertencia o crítica cuando la lectura supere el umbral	Operación principal	`fn_registrar_lectura` + `fn_clasificar_lectura`
R3	Calcular el promedio en tiempo real de un sensor en una ventana de minutos	Cálculo	`fn_promedio_sensor`
R4	Obtener estadísticas de un sensor en un periodo	Cálculo	`fn_estadisticas_sensor`
R5	Entregar a Grafana la serie de promedios por intervalo	Consulta	`fn_serie_promedios`
R6	Anticipar picos con la tendencia de las últimas horas	Cálculo	`fn_prediccion`
R7	Aplicar la misma regla de severidad en todas las funciones	Reutilización	`fn_clasificar_lectura`
R8	Los clientes solo ejecutan funciones; ninguno accede a las tablas	Seguridad	`SECURITY DEFINER` + `GRANT EXECUTE`

4. Diseño

4.1 Función principal: fn_registrar_lectura

La función sigue el principio de **validar primero y escribir después**: si cualquier validación falla, lanza una excepción con un código específico y no se escribe nada. Si todas pasan, inserta la lectura, la clasifica con la función reutilizable y, solo cuando es necesario, inserta la alerta. Como toda la función se ejecuta en una sola transacción, nunca queda una alerta sin su lectura.



Flujo interno de fn_registrar_lectura.

ELEMENTO	DEFINICIÓN
Firma	`fn_registrar_lectura(p_sensor TEXT, p_valor NUMERIC, p_medido_en TIMESTAMPTZ DEFAULT NOW())`
Retorno	`TABLE (lectura_id BIGINT, situacion TEXT)`: el identificador asignado y la clasificación
Lenguaje y volatilidad	PL/pgSQL, VOLATILE (modifica datos)
Errores	P0002 sensor inexistente · 55000 dispositivo no activo · 22003 valor fuera de rango
Seguridad	`SECURITY DEFINER` con `search_path` fijo; solo `consumidor_iot` puede ejecutarla

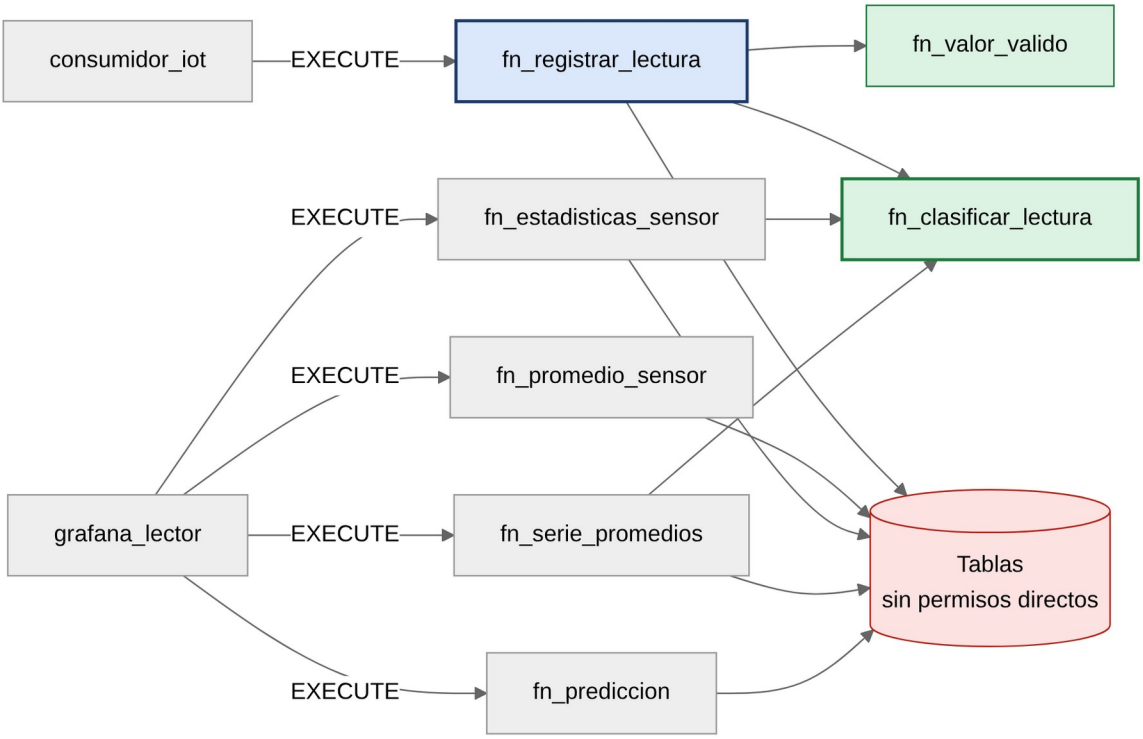
4.2 Funciones de apoyo

NO.	FUNCIÓN Y PARÁMETROS	RETORNO	TIPO
F1	`fn_clasificar_lectura(p_valor, p_umbral)`	TEXT	SQL · IMMUTABLE
F2	`fn_valor_valido(p_tipo, p_valor)`	BOOLEAN	SQL · IMMUTABLE
F4	`fn_promedio_sensor(p_sensor, p_minutos DEFAULT 10)`	NUMERIC	SQL · STABLE
F5	`fn_estadisticas_sensor(p_sensor, p_desde, p_hasta DEFAULT NOW(), 7 OUT)`	Registro	SQL · STABLE
F6	`fn_serie_promedios(p_tipo, p_desde, p_hasta, p_intervalo DEFAULT '1 minute')`	TABLE	SQL · STABLE
F7	`fn_prediccion(p_horas DEFAULT 3, p_horizonte DEFAULT 30)`	TABLE	SQL · STABLE
F8	`fn_promedio_sensor_ciclo(...)`: solo para el análisis de rendimiento	NUMERIC	PL/pgSQL · STABLE

`fn_prediccion` incorpora la retroalimentación recibida en la entrevista de la Tarea 7: ajusta una recta con `regr_slope` y `regr_intercept` sobre los promedios por minuto, mide la confianza con `regr_r2`, solo marca riesgo cuando la tendencia es clara e ignora los dispositivos en mantenimiento.

4.3 Reutilización y permisos

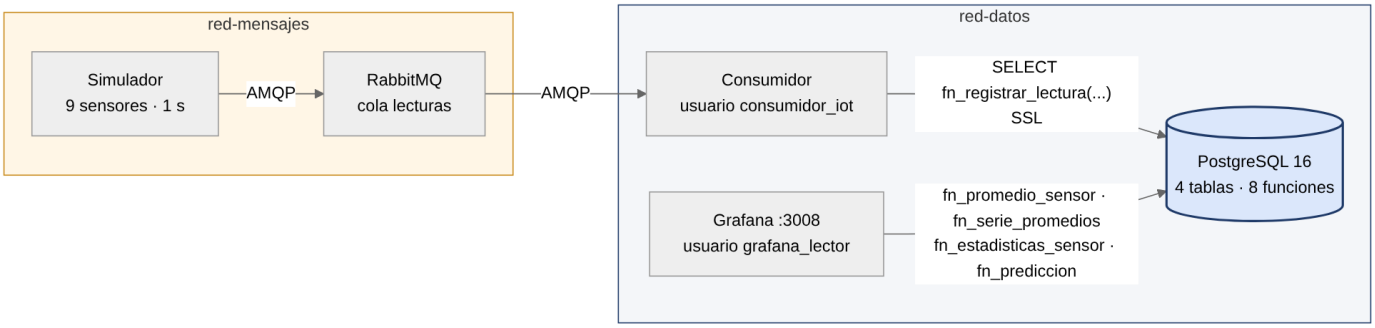
Las reglas de negocio están en funciones pequeñas que las demás llaman. Si NexoDev cambia la regla de severidad, se modifica solo `fn_clasificar_lectura` y se actualizan a la vez el registro de lecturas, las estadísticas y las gráficas. Los usuarios reciben `EXECUTE` solo sobre las funciones que necesitan; antes se quita el permiso que PostgreSQL da por defecto a todos (`REVOKE EXECUTE ... FROM PUBLIC`).



Reutilización entre funciones (verde) y permisos de cada usuario.

4.4 Arquitectura del entorno simulado

Cinco contenedores de Docker en dos redes. El simulador y RabbitMQ están en red-mensajes; PostgreSQL y Grafana en red-datos; el consumidor es el único en ambas. La base de datos no expone puertos a la PC y todas las conexiones usan SSL.



Arquitectura: las lecturas entran y se consultan siempre a través de funciones.

CONTENEDOR	IMAGEN	RED	PUERTO EN LA PC
funciones-datos	postgres:16 (SSL)	red-datos	Ninguno
funciones-rabbitmq	rabbitmq:3.13-management	red-mensajes	15678 (consola)
funciones-consumidor	iot-funciones-servicios:1.0 (Python 3.12)	red-datos y red-mensajes	Ninguno
funciones-simulador	iot-funciones-servicios:1.0 (Python 3.12)	red-mensajes	Ninguno
funciones-grafana	grafana/grafana:11.3.0	red-datos	3008

5. Implementación

5.1 Estructura del proyecto

```
iot-funciones/
├── docker-compose.yml        → base de datos, RabbitMQ, consumidor, simulador y Grafana
├── datos/
│   ├── init.sql              → tablas, 8 funciones, usuarios, permisos y datos de prueba
│   └── pruebas.sql           → pruebas de funcionamiento y rendimiento
├── servicios/
│   ├── Dockerfile, requirements.txt
│   ├── consumidor.py         → RabbitMQ → fn_registrar_lectura() (SQL embebido)
│   └── simulador.py          → 9 sensores IoT que publican en RabbitMQ
├── grafana/
│   ├── provisioning/         → origen de datos (grafana_lector, SSL) y tablero
│   └── dashboards/iot-funciones.json → 9 paneles, todos llaman funciones
└── docs/Documentacion_Tarea8.pdf
```

5.2 Buenas prácticas aplicadas

- **Nombres consistentes:** prefijo `fn_` para funciones y `p_` para parámetros, `v_` para variables locales.
- **Volatilidad correcta** (IMMUTABLE, STABLE, VOLATILE) para que el planificador optimice.
- **Operaciones por conjuntos** con funciones de agregado en lugar de ciclos fila por fila (ver sección 7).
- **Validación temprana** con `RAISE EXCEPTION` y códigos de error estándar que el cliente puede distinguir.
- **Seguridad:** `SECURITY DEFINER` siempre con `SET search_path`, `REVOKE ... FROM PUBLIC` y `GRANT EXECUTE` mínimo.
- **Consultas parametrizadas** desde Python: los valores nunca se concatenan al SQL, lo que evita inyección SQL.
- **Índices** que apoyan los filtros por sensor y por fecha de las funciones.

5.3 Base de datos y funciones: init.sql

```
-- ======================================================================
-- TAREA 8 · Programacion del lado del servidor: creacion de funciones
-- IoT Analytics Dashboard (NexoDev Solutions) · PostgreSQL 16
-- Las funciones encapsulan la logica reutilizable: registrar lecturas con
-- validacion y alertas, calcular promedios en tiempo real, estadisticas y
-- prediccion. Los usuarios solo ejecutan funciones; nadie toca las tablas.
-- ======================================================================

-- ------------------------------ TABLAS ------------------------------
CREATE TABLE dispositivos (
    id        SERIAL PRIMARY KEY,
    codigo    VARCHAR(20) NOT NULL UNIQUE,
    nombre    VARCHAR(60) NOT NULL,
    ubicacion VARCHAR(60) NOT NULL,
    estado    VARCHAR(15) NOT NULL DEFAULT 'activo'
              CHECK (estado IN ('activo', 'mantenimiento', 'inactivo'))
);

CREATE TABLE sensores (
    id             SERIAL PRIMARY KEY,
    dispositivo_id INT NOT NULL REFERENCES dispositivos(id),
    codigo         VARCHAR(20) NOT NULL UNIQUE,
    tipo           VARCHAR(20) NOT NULL CHECK (tipo IN ('temperatura', 'vibracion', 'energia')),
```

```
    unidad         VARCHAR(10) NOT NULL,
    umbral_max     NUMERIC(10,2) NOT NULL
);

CREATE TABLE lecturas (
    id        BIGSERIAL PRIMARY KEY,
    sensor_id INT NOT NULL REFERENCES sensores(id),
    valor     NUMERIC(10,2) NOT NULL,
    medido_en TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_lecturas_sensor_fecha ON lecturas (sensor_id, medido_en DESC);
CREATE INDEX idx_lecturas_fecha        ON lecturas (medido_en DESC);

CREATE TABLE alertas (
    id         SERIAL PRIMARY KEY,
    lectura_id BIGINT NOT NULL REFERENCES lecturas(id),
    sensor_id  INT NOT NULL REFERENCES sensores(id),
    severidad  VARCHAR(12) NOT NULL CHECK (severidad IN ('advertencia', 'critica')),
    valor      NUMERIC(10,2) NOT NULL,
    umbral     NUMERIC(10,2) NOT NULL,
    creada_en  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    atendida   BOOLEAN NOT NULL DEFAULT FALSE
);
CREATE INDEX idx_alertas_pendientes ON alertas (creada_en DESC) WHERE NOT atendida;

-- ============================== FUNCIONES ==============================

-- F1. Clasificar una lectura (funcion escalar pura).
-- IMMUTABLE: con los mismos parametros siempre devuelve lo mismo; el
-- planificador puede evaluarla una sola vez. La reutilizan F3, F5 y F6.
CREATE FUNCTION fn_clasificar_lectura(p_valor NUMERIC, p_umbral NUMERIC)
RETURNS TEXT
LANGUAGE sql IMMUTABLE PARALLEL SAFE AS $$
    SELECT CASE
        WHEN p_valor IS NULL               THEN 'sin datos'
        WHEN p_valor > p_umbral * 1.2      THEN 'critica'
        WHEN p_valor > p_umbral            THEN 'advertencia'
        ELSE 'normal'
    END;
$$;

-- F2. Rango fisico valido por tipo de sensor (funcion escalar con 3 parametros).
CREATE FUNCTION fn_valor_valido(p_tipo TEXT, p_valor NUMERIC)
RETURNS BOOLEAN
LANGUAGE sql IMMUTABLE PARALLEL SAFE AS $$
    SELECT CASE p_tipo
        WHEN 'temperatura' THEN p_valor BETWEEN -40 AND 150
        WHEN 'vibracion'   THEN p_valor BETWEEN 0 AND 50
        WHEN 'energia'     THEN p_valor BETWEEN 0 AND 1000
        ELSE FALSE
    END;
$$;

-- F3. Registrar una lectura (procedimiento con validacion y transaccion).
-- Valida sensor, estado del dispositivo y rango; inserta la lectura y,
-- si supera el umbral, la alerta. Devuelve una fila con el resultado.
-- SECURITY DEFINER: el consumidor solo tiene EXECUTE, no INSERT en tablas.
CREATE FUNCTION fn_registrar_lectura(
    p_sensor    TEXT,
    p_valor     NUMERIC,
    p_medido_en TIMESTAMPTZ DEFAULT NOW()
)
RETURNS TABLE (lectura_id BIGINT, situacion TEXT)
LANGUAGE plpgsql SECURITY DEFINER SET search_path = public AS $$
DECLARE
    v_sensor   sensores%ROWTYPE;
    v_estado   TEXT;
    v_id       BIGINT;
```

```
    v_sit      TEXT;
BEGIN
    SELECT s.* INTO v_sensor FROM sensores s WHERE s.codigo = p_sensor;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'El sensor % no existe', p_sensor USING ERRCODE = 'no_data_found';
    END IF;

    SELECT d.estado INTO v_estado FROM dispositivos d WHERE d.id = v_sensor.dispositivo_id;
    IF v_estado <> 'activo' THEN
        RAISE EXCEPTION 'El dispositivo de % esta en %', p_sensor, v_estado
              USING ERRCODE = 'object_not_in_prerequisite_state';
    END IF;

    IF NOT fn_valor_valido(v_sensor.tipo, p_valor) THEN
        RAISE EXCEPTION 'Valor % fuera del rango fisico de %', p_valor, v_sensor.tipo
              USING ERRCODE = 'numeric_value_out_of_range';
    END IF;

    INSERT INTO lecturas (sensor_id, valor, medido_en)
    VALUES (v_sensor.id, p_valor, p_medido_en)
    RETURNING id INTO v_id;

    v_sit := fn_clasificar_lectura(p_valor, v_sensor.umbral_max);
    IF v_sit IN ('advertencia', 'critica') THEN
        INSERT INTO alertas (lectura_id, sensor_id, severidad, valor, umbral, creada_en)
        VALUES (v_id, v_sensor.id, v_sit, p_valor, v_sensor.umbral_max, p_medido_en);
    END IF;

    RETURN QUERY SELECT v_id, v_sit;
END $$;

-- F4. Promedio en tiempo real de un sensor (escalar con parametro por defecto).
-- STABLE: no modifica datos; dentro de una consulta devuelve lo mismo.
-- Usa el indice (sensor_id, medido_en DESC): solo lee la ventana pedida.
CREATE FUNCTION fn_promedio_sensor(p_sensor TEXT, p_minutos INT DEFAULT 10)
RETURNS NUMERIC
LANGUAGE sql STABLE SECURITY DEFINER SET search_path = public AS $$
    SELECT ROUND(AVG(l.valor), 2)
    FROM lecturas l
    JOIN sensores s ON s.id = l.sensor_id
    WHERE s.codigo = p_sensor
      AND l.medido_en > NOW() - make_interval(mins => p_minutos);
$$;

-- F5. Estadisticas de un sensor en un periodo (parametros OUT -> un registro).
CREATE FUNCTION fn_estadisticas_sensor(
    p_sensor TEXT,
    p_desde  TIMESTAMPTZ,
    p_hasta  TIMESTAMPTZ DEFAULT NOW(),
    OUT lecturas      BIGINT,
    OUT promedio      NUMERIC,
    OUT minimo        NUMERIC,
    OUT maximo        NUMERIC,
    OUT desviacion    NUMERIC,
    OUT sobre_umbral  BIGINT,
    OUT situacion     TEXT
)
LANGUAGE sql STABLE SECURITY DEFINER SET search_path = public AS $$
    SELECT COUNT(*),
           ROUND(AVG(l.valor), 2),
           MIN(l.valor),
           MAX(l.valor),
           ROUND(STDDEV_SAMP(l.valor), 2),
           COUNT(*) FILTER (WHERE l.valor > s.umbral_max),
           fn_clasificar_lectura(MAX(l.valor), s.umbral_max)
    FROM sensores s
    LEFT JOIN lecturas l ON l.sensor_id = s.id
                        AND l.medido_en >= p_desde AND l.medido_en < p_hasta
```

```
    WHERE s.codigo = p_sensor
    GROUP BY s.umbral_max;
$$;

-- F6. Serie de promedios por intervalo (funcion de tabla para Grafana).
-- date_bin agrupa en cubetas del tamano pedido (1 min, 5 min, 1 h...).
CREATE FUNCTION fn_serie_promedios(
    p_tipo      TEXT,
    p_desde     TIMESTAMPTZ,
    p_hasta     TIMESTAMPTZ DEFAULT NOW(),
    p_intervalo INTERVAL DEFAULT '1 minute'
)
RETURNS TABLE (tiempo TIMESTAMPTZ, sensor TEXT, promedio NUMERIC, maximo NUMERIC, situacion TEXT)
LANGUAGE sql STABLE SECURITY DEFINER SET search_path = public AS $$
    SELECT date_bin(p_intervalo, l.medido_en, TIMESTAMPTZ '2000-01-01') AS tiempo,
           s.codigo::TEXT,
           ROUND(AVG(l.valor), 2),
           MAX(l.valor),
           fn_clasificar_lectura(MAX(l.valor), s.umbral_max)
    FROM lecturas l
    JOIN sensores s ON s.id = l.sensor_id
    WHERE s.tipo = p_tipo
      AND l.medido_en >= p_desde AND l.medido_en < p_hasta
    GROUP BY 1, 2, s.umbral_max
    ORDER BY 1, 2;
$$;

-- F7. Prediccion por regresion lineal (funcion de tabla con varios calculos).
-- Ajusta una recta a los promedios por minuto de las ultimas p_horas y
-- proyecta p_horizonte minutos. regr_r2 mide la confianza (0 a 1).
CREATE FUNCTION fn_prediccion(p_horas INT DEFAULT 3, p_horizonte INT DEFAULT 30)
RETURNS TABLE (sensor TEXT, actual NUMERIC, proyectado NUMERIC, umbral NUMERIC,
               pendiente_hora NUMERIC, confianza NUMERIC, minutos_al_umbral NUMERIC, riesgo TEXT)
LANGUAGE sql STABLE SECURITY DEFINER SET search_path = public AS $$
    WITH por_minuto AS (
        SELECT l.sensor_id,
               EXTRACT(EPOCH FROM date_trunc('minute', l.medido_en)) / 60 AS x,
               AVG(l.valor) AS y
        FROM lecturas l
        WHERE l.medido_en > NOW() - make_interval(hours => p_horas)
        GROUP BY 1, 2
    ), recta AS (
        SELECT sensor_id,
               regr_slope(y, x)     AS m,
               regr_intercept(y, x) AS b,
               regr_r2(y, x)        AS r2,
               MAX(x)               AS x_actual,
               COUNT(*)             AS puntos
        FROM por_minuto
        GROUP BY sensor_id
        HAVING COUNT(*) >= 10                        -- minimo de datos para predecir
    )
    SELECT s.codigo::TEXT,
           ROUND((r.m * r.x_actual + r.b)::NUMERIC, 2),
           ROUND((r.m * (r.x_actual + p_horizonte) + r.b)::NUMERIC, 2),
           s.umbral_max,
           ROUND((r.m * 60)::NUMERIC, 3),
           ROUND(COALESCE(r.r2, 0)::NUMERIC, 2),
           CASE WHEN r.m > 0 AND COALESCE(r.r2, 0) >= 0.3 AND (r.m * r.x_actual + r.b) <
s.umbral_max
                THEN ROUND(((s.umbral_max - (r.m * r.x_actual + r.b)) / r.m)::NUMERIC, 0) END,
           CASE WHEN COALESCE(r.r2, 0) < 0.3 THEN 'sin tendencia'
                WHEN r.m * (r.x_actual + p_horizonte) + r.b > s.umbral_max THEN 'riesgo de pico'
                WHEN r.m > 0 THEN 'subiendo'
                ELSE 'estable' END
    FROM recta r
    JOIN sensores s     ON s.id = r.sensor_id
    JOIN dispositivos d ON d.id = s.dispositivo_id
```

```
    WHERE d.estado = 'activo'                       -- sin falsas alarmas en mantenimiento
    ORDER BY s.codigo;
$$;

-- F8. Version ingenua de F4, solo para la prueba de rendimiento:
-- recorre las lecturas fila por fila con un ciclo en PL/pgSQL.
CREATE FUNCTION fn_promedio_sensor_ciclo(p_sensor TEXT, p_minutos INT DEFAULT 10)
RETURNS NUMERIC
LANGUAGE plpgsql STABLE SECURITY DEFINER SET search_path = public AS $$
DECLARE
    r      RECORD;
    v_suma NUMERIC := 0;
    v_n    INT := 0;
BEGIN
    FOR r IN SELECT l.valor, l.medido_en, s.codigo
             FROM lecturas l JOIN sensores s ON s.id = l.sensor_id LOOP
        IF r.codigo = p_sensor AND r.medido_en > NOW() - make_interval(mins => p_minutos) THEN
            v_suma := v_suma + r.valor;
            v_n := v_n + 1;
        END IF;
    END LOOP;
    RETURN CASE WHEN v_n > 0 THEN ROUND(v_suma / v_n, 2) END;
END $$;

-- ------------------------ USUARIOS Y PERMISOS ------------------------
-- Por defecto PostgreSQL deja ejecutar cualquier funcion a todos: se quita
REVOKE EXECUTE ON ALL FUNCTIONS IN SCHEMA public FROM PUBLIC;

-- Consumidor de RabbitMQ: solo puede registrar lecturas a traves de F3
CREATE ROLE consumidor_iot LOGIN PASSWORD 'consumidor123';
GRANT EXECUTE ON FUNCTION fn_registrar_lectura(TEXT, NUMERIC, TIMESTAMPTZ) TO consumidor_iot;

-- Grafana: solo funciones de consulta, ninguna tabla
CREATE ROLE grafana_lector LOGIN PASSWORD 'grafana123';
GRANT EXECUTE ON FUNCTION
    fn_clasificar_lectura(NUMERIC, NUMERIC),
    fn_promedio_sensor(TEXT, INT),
    fn_estadisticas_sensor(TEXT, TIMESTAMPTZ, TIMESTAMPTZ),
    fn_serie_promedios(TEXT, TIMESTAMPTZ, TIMESTAMPTZ, INTERVAL),
    fn_prediccion(INT, INT)
TO grafana_lector;

-- -------------------------- DATOS INICIALES --------------------------
INSERT INTO dispositivos (codigo, nombre, ubicacion) VALUES
('PRE-03', 'Prensa hidraulica 3', 'Linea 1'),
('TOR-01', 'Torno CNC 1',         'Linea 2'),
('EMB-02', 'Embotelladora 2',     'Linea 3');

INSERT INTO sensores (dispositivo_id, codigo, tipo, unidad, umbral_max) VALUES
(1, 'PRE03-TEMP', 'temperatura', '°C',   85.00),
(1, 'PRE03-VIB',  'vibracion',   'mm/s',  7.00),
(1, 'PRE03-ENE',  'energia',     'kWh',  30.00),
(2, 'TOR01-TEMP', 'temperatura', '°C',   80.00),
(2, 'TOR01-VIB',  'vibracion',   'mm/s',  6.00),
(2, 'TOR01-ENE',  'energia',     'kWh',  25.00),
(3, 'EMB02-TEMP', 'temperatura', '°C',   75.00),
(3, 'EMB02-VIB',  'vibracion',   'mm/s',  5.00),
(3, 'EMB02-ENE',  'energia',     'kWh',  20.00);

-- Historico de 3 dias (una lectura cada 30 s por sensor) para las pruebas de rendimiento.
-- PRE03-TEMP sube poco a poco en las ultimas 3 horas para probar la prediccion.
SELECT setseed(0.8);
INSERT INTO lecturas (sensor_id, valor, medido_en)
SELECT s.id,
       ROUND((s.umbral_max * (0.80 + (random() - 0.5) * 0.06)
              + CASE WHEN s.codigo = 'PRE03-TEMP' AND t > NOW() - INTERVAL '3 hours'
                     THEN EXTRACT(EPOCH FROM t - (NOW() - INTERVAL '3 hours')) / 3600 * 5
                     ELSE 0 END)::numeric, 2),
```

```
      t
FROM sensores s
CROSS JOIN generate_series(NOW() - INTERVAL '3 days', NOW() - INTERVAL '1 minute',
                           INTERVAL '30 seconds') AS t;
ANALYZE;
```

5.4 Consumidor: SQL embebido en Python

El consumidor no conoce las tablas: solo llama a `fn_registrar_lectura` con parámetros. Si la función rechaza la lectura, captura el error, registra su código y continúa con el siguiente mensaje.

```
"""
Consumidor: toma las lecturas de RabbitMQ y las registra llamando a la
funcion fn_registrar_lectura() con el usuario consumidor_iot, que solo tiene
permiso de EXECUTE sobre esa funcion. Toda la validacion y la generacion de
alertas vive en la base de datos.
"""
import json
import os
import time

import pika
import psycopg

CONNINFO = (f"host={os.getenv('DB_HOST', 'datos')} dbname=iotanalytics "
            f"user={os.getenv('DB_USER', 'consumidor_iot')} "
            f"password={os.environ['DB_PASSWORD']} sslmode=require")

REGISTRAR = "SELECT lectura_id, situacion FROM fn_registrar_lectura(%s::text, %s::numeric,
%s::timestamptz)"


def conectar_bd():
    while True:
        try:
            return psycopg.connect(CONNINFO, autocommit=True)
        except psycopg.OperationalError:
            print("PostgreSQL no disponible, reintentando...")
            time.sleep(3)


def conectar_rabbit():
    params = pika.URLParameters(os.getenv("RABBIT_URL", "amqp://iot:iot123@mensajes:5672/"))
    while True:
        try:
            return pika.BlockingConnection(params)
        except pika.exceptions.AMQPConnectionError:
            print("RabbitMQ no disponible, reintentando...")
            time.sleep(3)


def main():
    bd = conectar_bd()
    print("Conectado a PostgreSQL con SSL:", bd.info.pgconn.ssl_in_use)
    con = conectar_rabbit()
    canal = con.channel()
    canal.queue_declare(queue="lecturas", durable=True)
    canal.basic_qos(prefetch_count=50)
    cuenta = {"guardadas": 0, "alertas": 0, "rechazadas": 0}

    def al_recibir(ch, metodo, _props, cuerpo):
        lectura = json.loads(cuerpo)
        try:
            _id, situacion = bd.execute(REGISTRAR, (lectura["sensor"], lectura["valor"],
                                                    lectura["medido_en"])).fetchone()
            cuenta["guardadas"] += 1
```

```
            if situacion in ("advertencia", "critica"):
                cuenta["alertas"] += 1
                print(f"Alerta {situacion}: {lectura['sensor']} = {lectura['valor']}")
        except psycopg.Error as e:          # la funcion rechazo la lectura (RAISE EXCEPTION)
            cuenta["rechazadas"] += 1
            print(f"Rechazada [{e.sqlstate}]: {e.diag.message_primary}")
        ch.basic_ack(delivery_tag=metodo.delivery_tag)

    def reporte():
        print(f"{cuenta['guardadas']} guardadas · {cuenta['alertas']} alertas · "
              f"{cuenta['rechazadas']} rechazadas")
        con.call_later(30, reporte)

    con.call_later(30, reporte)
    canal.basic_consume(queue="lecturas", on_message_callback=al_recibir)
    print("Consumidor escuchando la cola 'lecturas'")
    canal.start_consuming()


if __name__ == "__main__":
    main()
```

5.5 Simulador de sensores

Publica una lectura por sensor cada segundo. Incluye picos de advertencia y críticos, una lectura defectuosa ocasional (-999) para probar las validaciones y una tendencia ascendente en PRE03-TEMP para probar la predicción.

```
"""
Simulador de sensores IoT: publica una lectura por sensor cada segundo
en la cola 'lecturas' de RabbitMQ. Nunca se conecta a la base de datos.
PRE03-TEMP sube poco a poco para que fn_prediccion() detecte la tendencia.
"""
import json
import os
import random
import time
from datetime import datetime, timezone

import pika

INTERVALO = float(os.getenv("INTERVALO", "1"))
# sensor -> (valor base, variacion, umbral)
SENSORES = {
    "PRE03-TEMP": (83, 1.5, 85),  "PRE03-VIB": (4.5, 0.6, 7), "PRE03-ENE": (22, 2, 30),
    "TOR01-TEMP": (65, 3, 80),  "TOR01-VIB": (3.8, 0.5, 6), "TOR01-ENE": (18, 2, 25),
    "EMB02-TEMP": (60, 3, 75),  "EMB02-VIB": (3.0, 0.4, 5), "EMB02-ENE": (14, 2, 20),
}


def conectar():
    params = pika.URLParameters(os.getenv("RABBIT_URL", "amqp://iot:iot123@mensajes:5672/"))
    while True:
        try:
            return pika.BlockingConnection(params)
        except pika.exceptions.AMQPConnectionError:
            print("RabbitMQ no disponible, reintentando...")
            time.sleep(3)


def main():
    con = conectar()
    canal = con.channel()
    canal.queue_declare(queue="lecturas", durable=True)
    print(f"Simulador publicando en la cola 'lecturas' cada {INTERVALO}s")
```

```
    enviados = 0
    inicio = time.time()
    while True:
        horas = (time.time() - inicio) / 3600
        for sensor, (base, variacion, umbral) in SENSORES.items():
            if sensor == "PRE03-TEMP":
                base += 5 * horas                    # tendencia: +5 °C por hora
            valor = random.gauss(base, variacion)
            r = random.random()
            if r < 0.02:
                valor = umbral * random.uniform(1.02, 1.15)   # pico: advertencia
            elif r < 0.03:
                valor = umbral * random.uniform(1.25, 1.40)   # pico: critica
            if random.random() < 0.002:
                valor = -999                         # lectura defectuosa: la funcion la rechaza
            mensaje = {"sensor": sensor, "valor": round(valor, 2),
                       "medido_en": datetime.now(timezone.utc).isoformat()}
            canal.basic_publish(exchange="", routing_key="lecturas", body=json.dumps(mensaje),
                                properties=pika.BasicProperties(delivery_mode=2))
            enviados += 1
        if enviados % 900 == 0:
            print(f"{enviados} lecturas publicadas")
        time.sleep(INTERVALO)


if __name__ == "__main__":
    main()
```

5.6 Orquestación con Docker Compose

```
# Tarea 8 · Creacion de funciones · IoT Analytics Dashboard (NexoDev Solutions)
name: iot-funciones

services:
  # ================= BASE DE DATOS (tablas + funciones) =================
  datos:
    image: postgres:16
    container_name: funciones-datos
    environment:
      POSTGRES_PASSWORD: ${DB_PASSWORD:-admin123}
      POSTGRES_DB: iotanalytics
    command: >
      -c ssl=on -c ssl_cert_file=/etc/ssl/certs/ssl-cert-snakeoil.pem
      -c ssl_key_file=/etc/ssl/private/ssl-cert-snakeoil.key
    volumes:
      - ./datos/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
    networks: [red-datos]          # sin puerto expuesto a la PC
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d iotanalytics"]
      interval: 3s
      retries: 30

  # ================= MENSAJERIA EN TIEMPO REAL =================
  mensajes:
    image: rabbitmq:3.13-management
    container_name: funciones-rabbitmq
    hostname: funciones-rabbitmq      # nombre fijo del nodo
    environment:
      RABBITMQ_DEFAULT_USER: iot
      RABBITMQ_DEFAULT_PASS: iot123
      # La cookie de Erlang va como argumento: RabbitMQ ya no lee
      # /var/lib/rabbitmq/.erlang.cookie, que falla con "eacces" en Docker Desktop
      RABBITMQ_SERVER_ADDITIONAL_ERL_ARGS: "-setcookie iotfuncionescookie"
    tmpfs:
      - /var/lib/rabbitmq:mode=1777
    ports:
      - "15678:15672"              # consola de RabbitMQ: http://localhost:15678
```

```
    networks: [red-mensajes]
    healthcheck:
      # Prueba que el puerto AMQP acepte conexiones (no usa la cookie)
      test: ["CMD-SHELL", "bash -c '</dev/tcp/localhost/5672'"]
      interval: 5s
      retries: 30

  # ========= CONSUMIDOR: RabbitMQ -> fn_registrar_lectura() =========
  consumidor:
    build: ./servicios
    image: iot-funciones-servicios:1.0
    container_name: funciones-consumidor
    command: python -u consumidor.py
    environment:
      DB_HOST: datos
      DB_USER: consumidor_iot        # solo puede ejecutar fn_registrar_lectura
      DB_PASSWORD: consumidor123
      RABBIT_URL: amqp://iot:iot123@mensajes:5672/
    networks: [red-datos, red-mensajes]
    depends_on:
      datos: { condition: service_healthy }
      mensajes: { condition: service_healthy }

  # ================= SIMULADOR DE SENSORES =================
  simulador:
    build: ./servicios
    image: iot-funciones-servicios:1.0
    container_name: funciones-simulador
    command: python -u simulador.py
    environment:
      RABBIT_URL: amqp://iot:iot123@mensajes:5672/
      INTERVALO: "1"
    networks: [red-mensajes]       # no tiene acceso a la base de datos
    depends_on:
      consumidor: { condition: service_started }

  # ================= VISUALIZACION =================
  grafana:
    image: grafana/grafana:11.3.0
    container_name: funciones-grafana
    environment:
      GF_SECURITY_ADMIN_PASSWORD: admin123
      GF_AUTH_ANONYMOUS_ENABLED: "true"
      GF_AUTH_ANONYMOUS_ORG_ROLE: Viewer
      GF_DASHBOARDS_DEFAULT_HOME_DASHBOARD_PATH: /var/lib/grafana/dashboards/iot-funciones.json
    ports:
      - "3008:3000"                # http://localhost:3008
    volumes:
      - ./grafana/provisioning:/etc/grafana/provisioning:ro
      - ./grafana/dashboards:/var/lib/grafana/dashboards:ro
    networks: [red-datos]
    depends_on:
      datos: { condition: service_healthy }

networks:
  red-datos:
  red-mensajes:
```

5.7 Integración con Grafana

El origen de datos usa el usuario `grafana_lector` con SSL. Los paneles llaman funciones y pasan como parámetros el rango de tiempo que el usuario elige en Grafana:

```
SELECT tiempo AS time, sensor AS metric, promedio AS value
FROM fn_serie_promedios('temperatura', $__timeFrom()::timestamptz, $__timeTo()::timestamptz, '1
minute')
```

La tabla de estadísticas llama a la misma función para cada sensor con `CROSS JOIN LATERAL`, un ejemplo de reutilización dentro de una consulta:

```
SELECT x.sensor, e.*
FROM unnest(ARRAY['PRE03-TEMP', 'PRE03-VIB', ...]) AS x(sensor)
CROSS JOIN LATERAL fn_estadisticas_sensor(x.sensor, NOW() - INTERVAL '1 hour') AS e
```

6. Pruebas y evidencias

Los datos de prueba son representativos de la operación real: 3 días de histórico con una lectura cada 30 segundos por sensor (cerca de 78,000 lecturas), más las lecturas en tiempo real del simulador. El script `datos/pruebas.sql` contiene las pruebas de funcionamiento y de rendimiento.

```
docker compose up -d --build
```

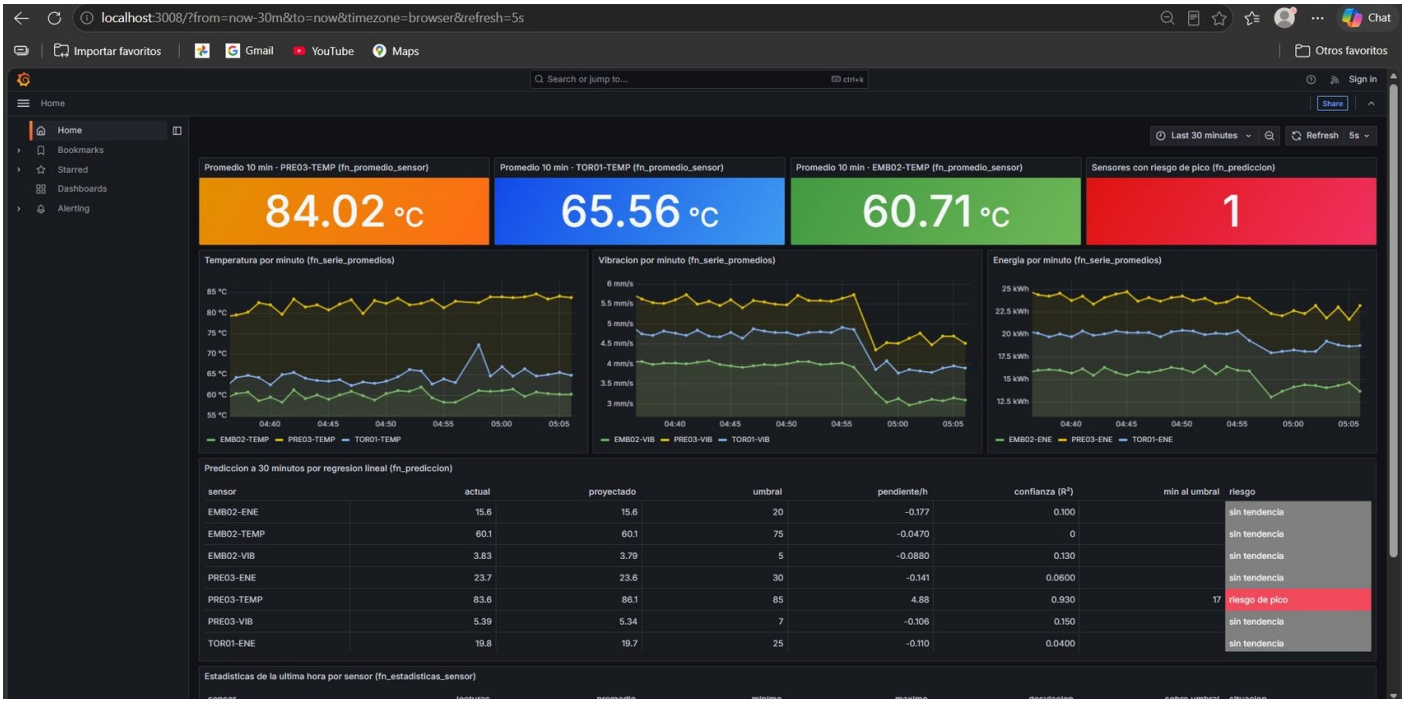
NO.	PRUEBA	RESULTADO ESPERADO	RESULTADO OBTENIDO
P1	Arranque del entorno	5 contenedores en ejecución	Correcto: datos y RabbitMQ healthy; consumidor, simulador y Grafana iniciados
P2	Integración con Grafana	Tablero alimentado solo por funciones	Correcto: promedios de 84.02, 65.56 y 60.71 °C; 1 sensor en riesgo de pico
P3	Creación de las funciones	8 funciones con sus parámetros y retornos	Correcto: `\df fn_*` lista las 8 con tipos escalar, record y TABLE
P4	Funciones escalares	70 normal, 90 advertencia, 105 crítica; vibración 99 inválida	Correcto: normal, advertencia, crítica y f (falso)
P5	Estadísticas (OUT)	Registro con 7 valores	Correcto: TOR01-TEMP 2,978 lecturas, promedio 64.09, máximo 110.65, situación crítica
P6	Predicción	Detectar la tendencia de PRE03-TEMP	Correcto: 83.07 → 85.49 °C, 4.84 °C/h, R^2 0.93, riesgo de pico en 24 min
P7	Registro válido	Guardar la lectura y su alerta	Correcto: lectura 79,493 registrada como crítica
P8	Validaciones	Rechazar sensor inexistente y valor fuera de rango	Correcto: “El sensor XX no existe”; “Valor 99 fuera del rango físico”
P9	Seguridad	El consumidor no puede insertar directo	Correcto: permission denied for table lecturas
P10	Integración en tiempo real	Lecturas por RabbitMQ registradas con la función	Correcto: 2,663 guardadas, 127 alertas, 10 rechazadas; 9 msg/s
P11	Rendimiento	SQL por conjuntos más rápido que el ciclo	Correcto: 314.9 ms contra 1.87 ms (≈168 veces menos)

Evidencia 1 — P1: los cinco contenedores del entorno simulado en ejecución.



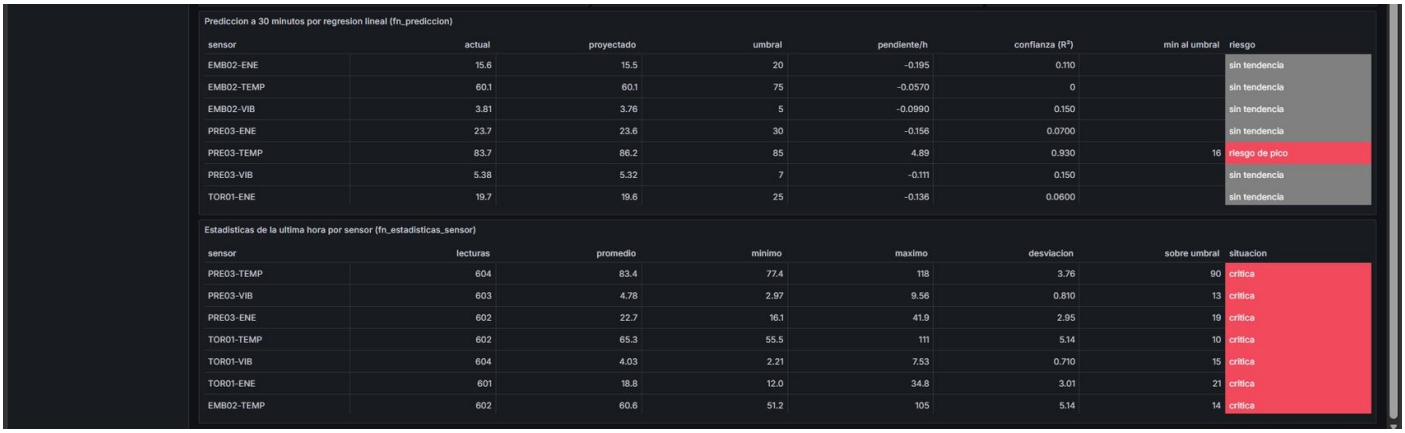
Arranque del entorno con Docker Compose.

Evidencia 2 — P2 y P6: el tablero de Grafana se construye llamando funciones: tres promedios en tiempo real con `fn_promedio_sensor`, las gráficas con `fn_serie_promedios` y la tabla de predicción con `fn_prediccion`, que marca a PRE03-TEMP en riesgo de pico.



Tablero IoT Analytics Dashboard alimentado por funciones.

Evidencia 3 — P5: la tabla inferior llama a `fn_estadisticas_sensor` para cada sensor. La columna situación aplica `fn_clasificar_lectura` al valor máximo del periodo, por eso todos los sensores aparecen en crítica: en la última hora cada uno tuvo al menos un pico mayor al umbral más 20 % generado por el simulador.



Prediccion a 30 minutos por regresion lineal (fn_prediccion)

sensor	actual	proyectado	umbral	pendiente/h	confianza (R^2)	min al umbral	riesgo
EMB02-ENE	15.6	15.5	20	-0.195	0.110		sin tendencia
EMB02-TEMP	60.1	60.1	75	-0.0570	0		sin tendencia
EMB02-VIB	3.81	3.76	5	-0.0990	0.150		sin tendencia
PRE03-ENE	23.7	23.6	30	-0.156	0.0700		sin tendencia
PRE03-TEMP	83.7	86.2	85	4.89	0.930	16	riesgo de pico
PRE03-VIB	5.38	5.32	7	-0.111	0.150		sin tendencia
TOR01-ENE	19.7	19.6	25	-0.136	0.0600		sin tendencia

Estadisticas de la ultima hora por sensor (fn_estadisticas_sensor)

sensor	lecturas	promedio	minimo	maximo	desviacion	sobre umbral	situacion
PRE03-TEMP	604	83.4	77.4	118	3.76	90	critica
PRE03-VIB	603	4.78	2.97	9.56	0.810	13	critica
PRE03-ENE	602	22.7	16.1	41.9	2.95	19	critica
TOR01-TEMP	602	65.3	55.5	111	5.14	10	critica
TOR01-VIB	604	4.03	2.21	7.53	0.710	15	critica
TOR01-ENE	601	18.8	12.0	34.8	3.01	21	critica
EMB02-TEMP	602	60.6	51.2	105	5.14	14	critica

Estadísticas de la última hora por sensor.

Evidencia 4 — P3: las 8 funciones creadas, con sus parámetros (incluidos DEFAULT y OUT) y sus tipos de retorno.

```
C:\Users\ASLHY\Downloads\iot-funciones\iot-funciones>
C:\Users\ASLHY\Downloads\iot-funciones\iot-funciones>docker exec funciones-datos psql -U postgres -d iotanalytics -c "\df fn_*"

                                                
```

```
Time: 1.871 ms
Timing is off.
=== Plan de la consulta de F4: usa el indice por sensor y fecha ===
                                                  QUERY PLAN
---------------------------------------------------------------------------------------------------------------
 Aggregate (actual time=0.726..0.728 rows=1 loops=1)
   ->  Hash Join (actual time=0.064..0.689 rows=119 loops=1)
         Hash Cond: (l.sensor_id = s.id)
         ->  Index Scan using idx_lecturas_fecha on lecturas l (actual time=0.020..0.364 rows=1071 loops=1)
               Index Cond: (medido_en > (now() - '00:10:00'::interval))
         ->  Hash (actual time=0.015..0.015 rows=1 loops=1)
               Buckets: 1024  Batches: 1  Memory Usage: 9kB
               ->  Seq Scan on sensores s (actual time=0.010..0.011 rows=1 loops=1)
                     Filter: ((codigo)::text = 'PRE03-VIB'::text)
                     Rows Removed by Filter: 8
 Planning Time: 0.443 ms
 Execution Time: 0.784 ms


C:\Users\ASLHY\Downloads\iot-funciones\iot-funciones>
```

EXPLAIN ANALYZE: lectura por índice en 0.78 ms.

Evidencia 7 — P7, P8 y P9: la función registra una lectura crítica, rechaza un sensor inexistente y un valor fuera de rango con mensajes claros, y el consumidor no puede insertar directo en la tabla.

```
C:\Users\ASLHY\Downloads\iot-funciones\iot-funciones>
C:\Users\ASLHY\Downloads\iot-funciones\iot-funciones>docker exec funciones-datos psql -U consumidor_iot -d iotanalytics -c "SELECT * FROM fn_registrar_lectu
ra('PRE03-TEMP', 110)"
 lectura_id | situacion
------------+-----------
      79493 | critica
(1 row)


C:\Users\ASLHY\Downloads\iot-funciones\iot-funciones>docker exec funciones-datos psql -U consumidor_iot -d iotanalytics -c "SELECT * FROM fn_registrar_lectu
ra('XX', 50)"
ERROR:  El sensor XX no existe
CONTEXT:  PL/pgSQL function fn_registrar_lectura(text,numeric,timestamp with time zone) line 10 at RAISE

C:\Users\ASLHY\Downloads\iot-funciones\iot-funciones>docker exec funciones-datos psql -U consumidor_iot -d iotanalytics -c "SELECT * FROM fn_registrar_lectu
ra('TOR01-VIB', 99)"
ERROR:  Valor 99 fuera del rango fisico de vibracion
CONTEXT:  PL/pgSQL function fn_registrar_lectura(text,numeric,timestamp with time zone) line 20 at RAISE

C:\Users\ASLHY\Downloads\iot-funciones\iot-funciones>docker exec funciones-datos psql -U consumidor_iot -d iotanalytics -c "INSERT INTO lecturas (sensor_id,
 valor) VALUES (1, 50)"
ERROR:  permission denied for table lecturas

C:\Users\ASLHY\Downloads\iot-funciones\iot-funciones>
```

Validaciones y seguridad de fn_registrar_lectura.

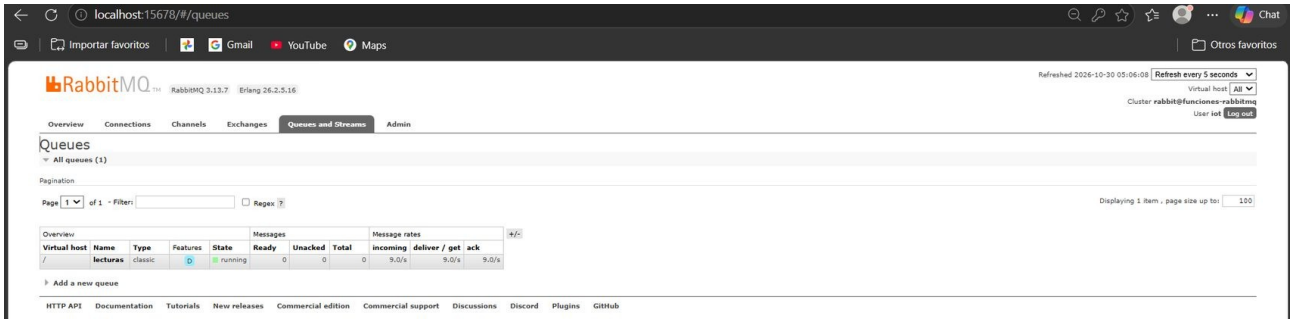
Evidencia 8 — P10: el consumidor registra cada lectura con la función; muestra las alertas generadas y una lectura defectuosa (-999) rechazada con el código 22003.

```
C:\Users\ASLHY\Downloads\iot-funciones\iot-funciones>docker compose logs --tail 10 consumidor
funciones-consumidor  | Alerta advertencia: EMB02-VIB = 5.39
funciones-consumidor  | Alerta critica: PRE03-TEMP = 117.78
funciones-consumidor  | Alerta advertencia: PRE03-TEMP = 85.6
funciones-consumidor  | Alerta advertencia: PRE03-TEMP = 85.14
funciones-consumidor  | Rechazada [22003]: Valor -999 fuera del rango fisico de temperatura
funciones-consumidor  | Alerta advertencia: PRE03-TEMP = 86.44
funciones-consumidor  | 2663 guardadas · 127 alertas · 10 rechazadas
funciones-consumidor  | Alerta advertencia: TOR01-ENE = 26.1
funciones-consumidor  | Alerta critica: EMB02-ENE = 27.69
funciones-consumidor  | Alerta advertencia: TOR01-VIB = 6.16

C:\Users\ASLHY\Downloads\iot-funciones\iot-funciones>
```

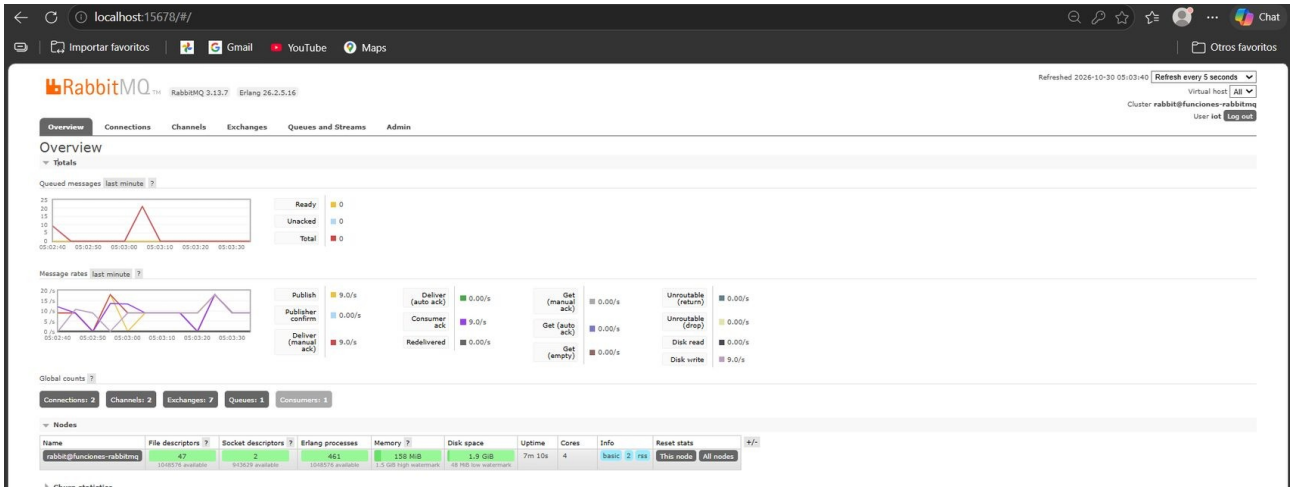
Registro del consumidor.

La cola `lecturas` de RabbitMQ entrega 9 mensajes por segundo sin acumularse.



Cola de RabbitMQ.

Vista general de RabbitMQ con 1 consumidor conectado.



Panel general de RabbitMQ.

7. Análisis de rendimiento

7.1 Ciclo fila por fila contra operación por conjuntos

Para medir el impacto del diseño se programó la misma operación de dos formas: `fn_promedio_sensor_ciclo` recorre todas las lecturas con un ciclo `FOR ... LOOP` en PL/pgSQL y acumula la suma, mientras que `fn_promedio_sensor` usa `AVG` con un filtro que aprovecha el índice. Ambas devuelven el mismo promedio (4.64).

VERSIÓN	CÓMO TRABAJA	FILAS QUE LEE	TIEMPO
`fn_promedio_sensor_ciclo`	Recorre toda la tabla y filtra dentro del ciclo	≈ 79,000	314.9 ms
`fn_promedio_sensor`	AVG por conjuntos con índice por fecha	≈ 1,071	1.87 ms

La versión por conjuntos fue **cerca de 168 veces más rápida**. La diferencia crece con el histórico: el ciclo tarda más cada día, mientras que la consulta con índice solo lee la ventana pedida.

7.2 Plan de ejecución

`EXPLAIN ANALYZE` muestra que PostgreSQL usa `Index Scan using idx_lecturas_fecha`: de unas 79,000 lecturas solo lee 1,071, las de los últimos 10 minutos, y termina en 0.78 ms. Después une con el sensor pedido y calcula el promedio.

7.3 Impacto en el IoT Analytics Dashboard

ASPECTO	SIN FUNCIONES	CON FUNCIONES
Validación de lecturas	En cada cliente, o no se hace	Una sola vez en `fn_registrar_lectura`: 10 lecturas defectuosas rechazadas en la prueba
Datos por la red	Grafana trae lecturas crudas	Grafana recibe promedios ya calculados
Consistencia de reglas	La severidad se repite en cada panel	Una sola regla: `fn_clasificar_lectura`
Seguridad	Clientes con permisos sobre tablas	Clientes con permiso solo de ejecutar
Escalabilidad	Más sensores = consultas más pesadas en cada cliente	Las funciones usan índices y se pueden optimizar en un solo lugar

La única desventaja medida es que `SECURITY DEFINER` impide que PostgreSQL integre (inline) las funciones SQL a la consulta que las llama. Se aceptó porque el costo es mínimo frente a la ventaja de que ningún cliente tenga permisos sobre las tablas.

8. Problemas encontrados y solución

8.1 La función no se encontraba desde Python

Al probar el consumidor, la llamada falló con “function fn_registrar_lectura(unknown, double precision, unknown) does not exist”. Python envía los números decimales como `double precision`, pero la función espera `NUMERIC`, y PostgreSQL busca la función por nombre **y por tipo de parámetros**.

Solución: convertir los parámetros explícitamente en la llamada. Este error mostró en la práctica que los tipos de los parámetros forman parte de la firma de una función.

```
REGISTRAR = "SELECT lectura_id, situacion FROM fn_registrar_lectura(%s::text, %s::numeric,
%s::timestamptz)"
```

8.2 Formato del indicador de Grafana

La tarjeta “Sensores con riesgo de pico” mostraba 1.00 porque heredaba el formato de 2 decimales de los promedios. Se fijó en 0 decimales y Grafana recargó el tablero automáticamente, sin reiniciar el contenedor.

8.3 Prevención: RabbitMQ

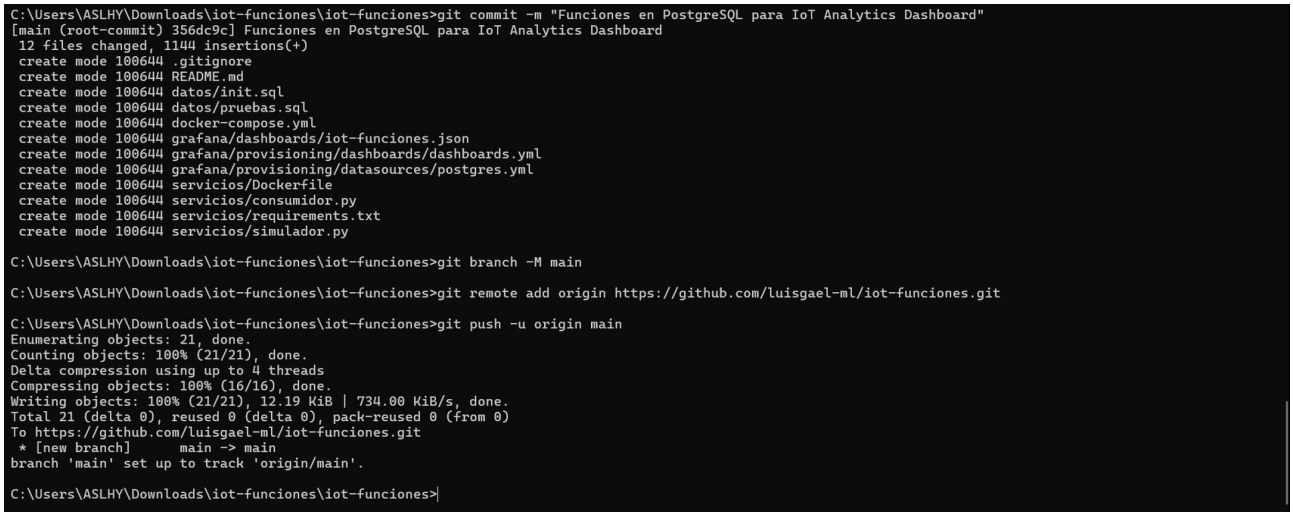
El error de permisos de `.erlang.cookie` de las Tareas 5 y 7 no se repitió, porque la solución documentada se incluyó desde el inicio en `docker-compose.yml`.

9. Control de versiones y colaboración

La actividad se desarrolló de forma individual. El proyecto se publicó en un repositorio público con un README que describe cada función, los usuarios y cómo levantar el entorno: https://github.com/luisgael-ml/iot-funciones

```
git init
git config user.name "Luis Gael Monterrubio López"
git config user.email "monterrubiolopezluisgael@gmail.com"
git add .
git commit -m "Funciones en PostgreSQL para IoT Analytics Dashboard"
git branch -M main
git remote add origin https://github.com/luisgael-ml/iot-funciones.git
git push -u origin main
```

Evidencia 9: el commit se registró a nombre de Luis Gael Monterrubio López y se publicó en GitHub.



Publicación del proyecto en GitHub.

9.1 Reflexión individual sobre la colaboración

En esta tarea busqué y apliqué retroalimentación de forma directa. En la entrevista de la Tarea 7 el CTO planteó cómo anticipar picos críticos, y propuse una regresión lineal con medida de confianza; en esta entrega la convertí en `fn_prediccion`, incluyendo los ajustes que surgieron al revisar la propuesta: la columna de confianza con `regr_r2` y excluir los equipos en mantenimiento.

También mejoré a partir de mis entregas anteriores: la solución del error de RabbitMQ que documenté en las Tareas 5 y 7 se incluyó desde el inicio y el problema ya no apareció. Las funciones facilitan el trabajo en equipo porque funcionan como un contrato: quien construye el tablero o el consumidor solo necesita conocer la firma de la función, sus parámetros y su retorno, mientras quien administra la base de datos puede mejorar la lógica interna sin romper a los demás. Por eso documenté cada firma en el README y en este informe.

10. Conclusiones

Las funciones permitieron llevar la lógica del IoT Analytics Dashboard al servidor, donde están los datos. `fn_registrar_lectura` concentra en una llamada la validación, el registro y la generación de alertas, y garantiza que ninguna lectura imposible o de un equipo en mantenimiento llegue a la base de datos, sin importar qué cliente la envíe.

Los parámetros con valores por defecto y de salida, junto con los distintos tipos de retorno (escalar, registro y tabla), hicieron que cada función se adapte a quien la usa: Grafana recibe series y tablas listas para graficar, y el consumidor recibe la situación de cada lectura. La reutilización de `fn_clasificar_lectura` asegura que la regla de severidad sea la misma en todo el sistema.

El análisis de rendimiento mostró que la forma de programar una función importa tanto como la función misma: la versión por conjuntos con índice fue cerca de 168 veces más rápida que el ciclo fila por fila. Con permisos de solo ejecución, el sistema quedó más seguro, eficiente y escalable, cumpliendo el objetivo de NexoDev.

Referencias

Docker Inc. (s.f.). *Networking in Compose*. Docker Docs. https://docs.docker.com/compose/how-tos/networking/

Grafana Labs. (s.f.). *PostgreSQL data source*. Grafana documentation. https://grafana.com/docs/grafana/latest/datasources/postgres/

PostgreSQL Global Development Group. (s.f.). *Aggregate functions (regr_slope, regr_r2)*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/functions-aggregate.html

PostgreSQL Global Development Group. (s.f.). *CREATE FUNCTION*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/sql-createfunction.html

PostgreSQL Global Development Group. (s.f.). *Errors and messages (RAISE)*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/plpgsql-errors-and-messages.html

PostgreSQL Global Development Group. (s.f.). *Function volatility categories*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/xfunc-volatility.html

PostgreSQL Global Development Group. (s.f.). *PL/pgSQL: SQL procedural language*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/plpgsql.html

PostgreSQL Global Development Group. (s.f.). *Query language (SQL) functions*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/xfunc-sql.html

Psycopg. (s.f.). *Passing parameters to SQL queries*. Psycopg 3 documentation. https://www.psycopg.org/psycopg3/docs/basic/params.html